B18 Documentation and communication

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2
B18 Documentation and communication 3
B18-L1 Docstrings and READMEs that answer the next person's questions 3
B18-L2 Model cards that state limits as clearly as scores 7
B18-L3 Decision logs, so the reasons outlive the meeting 10
Track B best-practice checklist 15
Appendix: solutions and answer keys 16

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B18 Documentation and communication

Roadmap: Prerequisite for all months; Every month. Docstrings, READMEs, model cards and decision logs.

B18-L1 Docstrings and READMEs that answer the next person's questions

Concept

The next person to read your code, often you in six months, has three questions: what does this do, how do I run it, and what will surprise me? Docstrings answer them for a function. The README answers them for the project.

Docstrings. This course uses the Google style (CLAUDE.md), which tools such as Sphinx and mkdocstrings read:

```
def weighted_share(df: pd.DataFrame, value: str, weight: str) -> float | None:
    """Weighted mean of a 0/1 column. Grain: one row per respondent.

    Args:
        df: Respondent-level data.
        value: 0/1 column to average.
        weight: Non-negative weight column. Rows with a missing value or weight are dropped.

    Returns:
        The weighted share, or None when no rows remain or weights sum to 0.

    Raises:
        KeyError: If a named column is missing.
    """
```

Put in the docstring what the signature cannot say: the **grain**, **units**, how **missing values** and **zeros** are handled, and what `None` means. Leave out what the code already says plainly, such as "loops over the rows".

`help(weighted_share)` and `weighted_share.__doc__` show the docstring. Editors show it on hover.

README. One page at the project root, in this order:

1. **Purpose**: one paragraph, including who uses the output.

2. **Quick start**: the exact commands to install and run, copied from a working terminal.
3. **Inputs and outputs**: sources, tables or files, their grain and schedule.
4. **Configuration**: where settings live and which ones matter.
5. **Tests and checks**: how to run them, and what CI enforces.
6. **Owners and support**: a team, not a person who might leave.

Keep both close to the truth. Update the docstring in the same commit as the code, and test the README's quick start in CI where you can.

Why it matters

A weighting function returned `None` for empty cells. Its docstring said "Returns the weighted share." A dashboard developer read that, assumed a float, and wrote `round(share * 100)`. The first empty cell crashed the nightly refresh. One line in the docstring, "or None when no rows remain", would have told them to handle it.

Worked example: weak code

```
1 def calc(df, v, w):
2     # calculate
3     d = df[[v, w]].dropna()
4     return (d[v] * d[w]).sum() / d[w].sum() if d[w].sum() else None
```

1. **Line 1.** Name, parameters and types say nothing about purpose or inputs.
2. **Line 2.** A comment that restates the obvious and hides what matters.

Worked example: strong code

```
 1 import pandas as pd
 2
 3
 4 def weighted_share(df: pd.DataFrame, value: str, weight: str) -> float |
     None:
 1     """Weighted mean of a 0/1 column. Grain: one row per respondent.
 6
 7     Args:
 8         df: Respondent-level data.
 9         value: 0/1 column to average.
10         weight: Non-negative weight column. Rows missing value or weight
    are dropped.
11
12     Returns:
 2         The weighted share, or None when no rows remain or weights sum to
    0.
14
15     Raises:
 3         KeyError: If a named column is missing.
```

```
    """
    d = df[[value, weight]].dropna()
    total = d[weight].sum()
    if len(d) == 0 or total == 0:
        return None
    return float((d[value] * d[weight]).sum() / total)
```

1. **Line 5.** One-line summary with the grain.
2. **Line 13.** What None means, so callers handle it.
3. **Line 16.** The failure a caller might need to catch.

Docstrings state grain, units, missing values and what None means. The README states purpose, quick start, inputs, config, tests and owners.

Recap card

- **Google-style docstring sections** One-line summary, then Args:, Returns:, Raises:.
- **What a data docstring must state** Grain, units, missing and zero handling, and what None means.
- **README sections** Purpose, quick start, inputs and outputs, configuration, tests, owners.
- **Keeping docs true** Update docstrings in the same commit as the code. Test the quick start where possible.

Practice

Exercise 1. Multiple choice (level 1)

Which line adds the most value to a function's docstring?

- a) "Loops over rows and sums values."
- b) "Returns None when the weights sum to 0."
- c) "Written by the analytics team."
- d) "This is a function."

Why?

- r1) It tells callers about a behaviour the signature and name cannot show.
- r2) Short lines are better.

Exercise 2. Predict the output (level 2)

What does this print?

```
def share(part: float, total: float) -> float | None:
    """part / total, or None when total is 0."""
    return None if total == 0 else part / total
```

```
4
5
6 print(share.__doc__)
```

Exercise 3. Put in order (level 2)

Order the sections of a project README.

- Tests and checks
- Inputs and outputs, with grain and schedule
- Owners and support
- Quick start: install and run commands
- Purpose and users
- Configuration

Exercise 4. Write the function (level 3)

Add a Google-style docstring to `cap_weights`. It must have a one-line summary, an `Args:` section naming `weights` and `cap`, a `Returns:` section, and a `Raises:` section with `ValueError` for a cap below 1. Mention that missing weights stay missing. Keep the code as it is.

Your code must pass these checks:

- Include Args:, Returns: and Raises: sections.
- Name both parameters under Args:, and ValueError under Raises:.
- Say what happens to missing weights.
- Start with a one-line summary.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
1 # Cap survey weights at a maximum value.
2 def cap_weights(weights: pd.Series, cap: float) -> pd.Series: ...
```

Starter code:

```
1 import pandas as pd
2
3
4 def cap_weights(weights: pd.Series, cap: float) -> pd.Series:
5     if cap < 1:
6         raise ValueError("cap must be at least 1")
7     return weights.clip(upper=cap)
```

Exercise 5. Architecture design (level 4)

Decide where each piece of information belongs.

Options: Docstring, README, Inline comment, Config file.

1. The function returns None for empty cells
2. The command to run the monthly job
3. Why one line uses a workaround for a library bug, with the issue link
4. The minimum reporting base of 50

B18-L2 Model cards that state limits as clearly as scores

Concept

A **model card** is a short document that travels with a model. It tells users what the model is for, how well it works, for whom, and where it should not be used. The format comes from Mitchell et al., "Model Cards for Model Reporting" (2019). Keep it in the repository next to the training code, and link it from the registered model version (lesson B14-L2).

Sections to include:

- **Model details**: name, version, registry alias, owners, training date, run ID.
- **Intended use**: the decision it supports and who uses it.
- **Out-of-scope uses**: uses it must not serve, stated plainly.
- **Training data**: source, period, grain, known gaps.
- **Evaluation**: the honest metric (lesson B11-L3), with the test period and prevalence.
- **Subgroup results**: rates per group with counts (lesson B13-L3), including the weakest group.
- **Limitations and risks**: known failure modes, drift risks, proxies.
- **Monitoring and review**: what is tracked, alert thresholds, next review date.

Write numbers with their context: "AUC 0.78 on October to December 2026 customers, 2.4% positive rate, 51,000 rows". A bare "AUC 0.78" invites misuse.

State limits as clearly as strengths. "Recall for customers over 70 is 0.28 against 0.62 overall. Do not use for outreach targeting of this group without human review" protects the people affected and the team that built the model.

Treat the card as code: a template with required sections, reviewed in the same pull request as the model change. A check in CI can refuse a card with missing sections.

Why it matters

A propensity model built for marketing emails was later reused by another team to prioritise collections calls. Nobody had written down that it was trained only on customers who had opted in to marketing, a group with very different payment behaviour. Collections priorities were wrong for months. An "Out-of-scope uses" line, "Not validated for credit or collections decisions", would have stopped the reuse at the first review.

Worked example: weak code

```
Churn model v4
AUC: 0.78
Owner: Sam
Works well, use for retention.
```

1. **Line 2.** No test period, prevalence or subgroup results.
2. **Line 3.** A person, not a team. Sam may leave.
3. **Line 4.** No limits, no out-of-scope uses, no monitoring.

Worked example: strong code

```
Model details: churn_model v4 (@champion), owner Retention Analytics, run 4da8..., trained 2026-09-01
Intended use: rank active customers for the monthly retention call list (top 500)
Out of scope: credit, collections or pricing decisions; customers with under 3 months' tenure
Training data: CRM extract Jan 2024 to Jun 2026, one row per customer-month
Evaluation: AUC 0.78, average precision 0.21 (prevalence 2.4%), test Oct to Dec 2026, 51,000 rows
Subgroups: recall at top 500 is 0.62 overall, 0.28 for customers over 70 (n positives = 64)
Limitations: sparse digital activity lowers recall for older customers; tenure proxies contract type
Monitoring: monthly AUC and calibration, alert below 0.72; next review 2027-03-01
```

1. **Line 3.** Stops reuse the model was never validated for.
2. **Line 5.** Each number with its period, baseline and size.
3. **Line 6.** The weakest group, stated with its count.

A model card states purpose, limits, data, honest metrics, subgroup results and monitoring, and is reviewed with every model change.

Recap card

- **Model card sections** Details, intended use, out-of-scope uses, training data, evaluation, subgroups, limitations, monitoring.
- **How to report a metric on a card** With test period, prevalence and row count.
- **Why out-of-scope uses** They stop reuse for decisions the model was never validated for.
- **Keep cards honest** Template with required sections, reviewed in the model's pull request, checked in CI.

Practice

Exercise 1. Multiple choice (level 1)

Which model card section would most directly have prevented a marketing model being reused for collections?

- a) Model details
- b) Out-of-scope uses
- c) Evaluation
- d) Monitoring

Why?

- r1) It names the uses the model was not built or validated for.
- r2) It is the longest section.

Exercise 2. Predict the output (level 2)

What does this print?

```
REQUIRED = ["intended_use", "out_of_scope", "evaluation", "subgroups",
    "limitations"]
card = {"intended_use": "retention list", "evaluation": "AUC 0.78",
    "limitations": ""}
print([s for s in REQUIRED if not str(card.get(s, "")).strip()])
```

Exercise 3. Fill the gap (level 2)

Complete the evaluation line so the metric has its context.

```
Evaluation: AUC 0.78, average precision 0.21 (_____(1) 2.4%), test Oct to
    Dec 2026, 51,000 _____(2)
```

Exercise 4. Write the function (level 3)

Write `missing_sections(card: dict[str, str], required: list[str]) -> list[str]`. Return the required section names, in the order of `required`, that

are absent from `card` or contain only whitespace. CI will fail the build when the list is not empty.

Your code must pass these checks:

- Absent sections must be listed.
- Sections with only whitespace count as missing.
- Keep the order of required.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Required model card sections that are absent or blank.
def missing_sections(
    card: dict[str, str],
    required: list[str],
) -> list[str]: ...
```

Starter code:

```python
def missing_sections(card, required):
    return [s for s in required if s not in card]
```

Exercise 5. Architecture design (level 4)

Write the subgroup and limitations parts of a card for a hardship-support model.

Options: Recall and false positive rate per age band, with counts, weakest group first, Overall recall only.

1. Subgroup results
2. Known weakness for customers over 70
3. Review

Sources

- The model card format comes from Mitchell et al. (2019), Model Cards for Model Reporting. (verified: Mitchell et al., Proceedings of FAT* 2019, pages 220 to 229.)

B18-L3 Decision logs, so the reasons outlive the meeting

Concept

Every analysis rests on choices: the weighting scheme, the outlier rule, the metric, the cut-off, the fairness property. Six months later someone asks

"why did we do it this way?". Without a record, the answer is a guess, and the choice gets reversed for the wrong reasons or kept for no reason.

A **decision log** is a list of short, dated entries, one per decision, kept in the repository (for example `docs/decisions/`). The format follows software architecture decision records:

```
## D-007: Trim survey weights at 5
Date: 2026-09-12. Status: accepted.
Context: Four respondents had weights above 12 and moved brand share by 4 points.
Options considered:
- No trimming
- Trim at 5, redistribute to keep the weighted total
- Rake with bounded weights
Decision: Trim at 5 with redistribution. Raking deferred until the new panel design.
Consequences: Design effect falls from 2.1 to 1.6. Small bias towards the sample profile. Review at wave 20.
```

Rules:

- **Write it when you decide**, in the same pull request as the code or config change.
- **List real options**, at least two, including "do nothing". A decision with one option was not a decision.
- **State the consequences**, including the costs you accept.
- **Never edit a past decision's reasoning.** When things change, add a new entry and mark the old one `superseded`, pointing to its replacement. The history of thinking is the value.
- **Link it**: from the config value it explains, from the model card (lesson B18-L2) and from the assumptions log (lesson B7-L3).

Keep entries short. Five to fifteen lines is enough for most analytical choices. A long, perfect document written after the fact helps less than a short one written on the day.

Why it matters

A new lead analyst removed the weight trimming from a tracker because it "looked arbitrary". The next wave's brand share jumped 5 points on four heavily weighted respondents, and the client asked why the trend broke. The original team had discussed exactly this case in a meeting two years earlier. Nothing was written down. A three-line decision entry linked from the config would have explained the rule in a minute.

Worked example: weak code

```
# config.yaml
weight_cap: 5   # don't change!!
```

1. **Line 2.** A warning with no reason invites either blind obedience or careless removal.

Worked example: strong code

```
# config.yaml
weight_cap: 5   # D-007, docs/decisions/D-007-trim-weights.md

# docs/decisions/D-007-trim-weights.md
## D-007: Trim survey weights at 5
Date: 2026-09-12. Status: accepted.
Options considered: no trimming; trim at 5 with redistribution; bounded
  raking.
Decision: trim at 5 with redistribution.
Consequences: design effect 2.1 to 1.6; review at wave 20.
```

1. **Line 2.** The value links to its reasons.
2. **Line 7.** Real alternatives, including doing nothing.
3. **Line 9.** The cost accepted, and when to revisit.

One short, dated entry per decision, with options, consequences and status, linked from the code it explains, and superseded rather than edited.

Recap card

- **Parts of a decision entry** ID and title, date and status, context, options considered, decision, consequences.
- **When a decision changes** Add a new entry. Mark the old one superseded and link to the new one.
- **Where and when to write** In the repository, in the same pull request as the change.
- **Make it findable** Link from the config value, the model card and the assumptions log.

Practice

Exercise 1. Multiple choice (level 1)

A decision from last year no longer holds. What should you do with its log entry?

- a) Edit it to describe the new decision.
- b) Delete it.

- c) Add a new entry, and mark the old one superseded with a link to the new one.
- d) Leave both without links.

Why?

- r1) The log keeps the history of reasoning. Each entry records what was known when it was written.
- r2) Logs must never shrink for storage reasons.

Exercise 2. Predict the output (level 2)

What does this print?

```
log = {
    "D-003": {"status": "superseded", "superseded_by": "D-007"},
    "D-007": {"status": "accepted"},
}
current = [k for k, v in log.items() if v["status"] == "accepted"]
print(current, log["D-003"]["superseded_by"] in log)
```

Exercise 3. Put in order (level 2)

Order the parts of a decision entry.

- Decision
- ID and title
- Consequences and review point
- Context: the problem and the evidence
- Date and status
- Options considered

Exercise 4. Write the function (level 3)

Write `decision_problems(entry: dict[str, object], log_ids: set[str]) -> list[str]` that checks one decision entry and returns problem messages, in this order:

1. `"status"` must be one of `"proposed"`, `"accepted"`, `"superseded"`. Otherwise add `"invalid status"`.
2. `"options"` must be a list with at least 2 items. Otherwise add `"fewer than two options"`.
3. `"consequences"` must be non-blank text. Otherwise add `"no consequences"`.
4. If status is `"superseded"`, `"superseded_by"` must be in `log_ids`. Otherwise add `"superseded without a valid replacement"`.

Your code must pass these checks:

- A complete accepted entry has no problems.
- Check status, options (at least 2) and non-blank consequences, one message each.
- A superseded entry must point to an ID that exists in the log.
- Missing keys count as problems, reported in the rule order.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
VALID_STATUS = ...

# Problems with one decision log entry, in rule order.
def decision_problems(
    entry: dict[str, object],
    log_ids: set[str],
) -> list[str]: ...
```

Starter code:

```python
def decision_problems(entry, log_ids):
    return []
```

Exercise 5. Architecture design (level 4)

Decide how the team handles decision records.

Options: docs/decisions/ in the repository, one file per decision, Slides from the steering meeting.

1. Where entries live
2. When an entry is written
3. How readers find it

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B18-L1 Docstrings and READMEs that answer the next person's questions

B18-L1 exercise 1

Hints. (1) What would surprise a caller? (2) What can the signature not say? (3) Edge behaviour.

Answer. b) "Returns None when the weights sum to 0." Because: r1) It tells callers about a behaviour the signature and name cannot show.

Document what a caller must know and cannot see: edge cases, units, grain.

Common wrong answers.

- `a`: The code already shows that. Docstrings explain what the code cannot.
- `c`: Ownership belongs in the README or CODEOWNERS, not every docstring.
- `d`: It adds nothing.
- `b|r2`: Right answer, wrong reason. Value comes from information, not length.

B18-L1 exercise 2

Hints. (1) Where does Python store a function's docstring? (2) In **doc**. (3) It prints the string.

Answer.

```
part / total, or None when total is 0.
```

The first string in the body is the docstring. `help()`, editors and documentation tools read it from `__doc__`.

Common wrong answers.

- `None`: A string literal as the first statement of a function becomes its **doc**.

B18-L1 exercise 3

Hints. (1) What does a newcomer need first? (2) Then how to run it. (3) Details after.

Answer.

1. Purpose and users
2. Quick start: install and run commands
3. Inputs and outputs, with grain and schedule
4. Configuration
5. Tests and checks
6. Owners and support

Purpose, run, data, settings, checks, contacts: the order a newcomer's questions arrive in.

Common wrong answers.

- Say what the project is before telling people to run it.
- The quick start is what most readers came for. Put it near the top.

B18-L1 exercise 4

Hints. (1) Summary line, blank line, then Args:, Returns:, Raises:. (2) Indent each entry under its section. (3) Mention missing weights under weights.

Answer.

```python
import pandas as pd


def cap_weights(weights: pd.Series, cap: float) -> pd.Series:
    """Cap survey weights at a maximum value.

    Args:
        weights: One weight per respondent. Missing weights stay missing.
        cap: Largest allowed weight, at least 1.

    Returns:
        A new Series with every weight above cap set to cap.

    Raises:
        ValueError: If cap is below 1.
    """
    if cap < 1:
        raise ValueError("cap must be at least 1")
    return weights.clip(upper=cap)
```

The docstring now tells a caller the bounds, the edge behaviour and the failure mode, none of which the signature shows.

B18-L1 exercise 5

Hints. (1) Who reads it, and where? (2) Will the value change? (3) How local is the explanation?

Answer.

- Behaviour a caller must handle: the docstring.
- How to run the project: the README.
- Why a specific line is unusual: an inline comment next to it.
- A setting that may change: config (lesson A1-L3), described in the README.

Put information where its reader looks, and settings where they can change without editing text.

Common wrong answers.

- none_meaning = `readme`: Callers read the function's docstring, often in their editor, not the README.
- threshold = `docstring`: A value that can change belongs in config. Hardcoding it in prose invites drift.
- why_line = `docstring`: A workaround on one line is explained best right beside that line.

B18-L2 Model cards that state limits as clearly as scores

B18-L2 exercise 1

Hints. (1) Which section says 'do not use for'? (2) Reuse is a scope question. (3) Out of scope.

Answer. b) Out-of-scope uses Because: r1) It names the uses the model was not built or validated for.

Out-of-scope uses turn assumptions in the builders' heads into a rule readers see.

Common wrong answers.

- `a`: Details identify the model. They do not say where it must not be used.
- `c`: Evaluation shows how it performs on its intended task, not on others.
- `d`: Monitoring tracks the model in use. It does not stop misuse.
- `b|r2`: Right answer, wrong reason. Length does not matter. Its purpose does.

B18-L2 exercise 2

Hints. (1) Which keys are absent? (2) Which are present but blank? (3) Both count.

Answer.

```
['out_of_scope', 'subgroups', 'limitations']
```

Empty sections are as unhelpful as missing ones. A CI check should reject both.

Common wrong answers.

- `['out_of_scope', 'subgroups']`: limitations exists but is empty. The check treats blank text as missing.

B18-L2 exercise 3

Hints. (1) The baseline for average precision. (2) How much evidence stands behind the number? (3) Prevalence and row count.

Answer.

```
Evaluation: AUC 0.78, average precision 0.21 (prevalence 2.4%), test Oct to Dec 2026, 51,000 rows
```

Prevalence gives the baseline and the row count gives the evidence behind the metric.

Common wrong answers.

- g1 = `accuracy`: Accuracy is another metric. Average precision needs the prevalence as its baseline.

B18-L2 exercise 4

Hints. (1) Loop over required, not over card. (2) card.get(name, "") handles absent keys. (3) Use .strip() to treat whitespace as empty.

Answer.

```python
def missing_sections(card: dict[str, str], required: list[str]) -> list[str]:
    """Required model card sections that are absent or blank.

    Args:
        card: Section name to text.
        required: Section names that must have content.

    Returns:
        Missing section names, in the order of required.
    """
    return [name for name in required if not card.get(name, "").strip()]
```

A small, strict check turns the template into a gate: no model ships with an empty limitations section.

B18-L2 exercise 5

Hints. (1) What must users know to use the model safely? (2) What happens to hidden weaknesses? (3) Who reviews, and when?

Answer.

- Per-group rates with counts, so the weakest group is visible.
- The gap stated plainly with a mitigation.
- A team and a date that make review happen.

A card is useful when it makes weaknesses and responsibilities impossible to miss.

Common wrong answers.

- subgroups = `overall`: An overall rate hides a group at half the recall (lesson B12-L3).
- limit = `omit`: Hiding a known weakness moves the harm onto the people the model serves.
- review = `adhoc`: Without a date and an owner, reviews do not happen.

B18-L3 Decision logs, so the reasons outlive the meeting

B18-L3 exercise 1

Hints. (1) Why keep past reasoning? (2) How would a reader know it no longer applies? (3) Supersede and link.

Answer. c) Add a new entry, and mark the old one superseded with a link to the new one. Because: r1) The log keeps the history of reasoning. Each entry records what was known when it was written.

Supersede, never rewrite. The chain of entries shows how and why thinking changed.

Common wrong answers.

- `a`: Editing erases why the first choice made sense at the time.
- `b`: Deleting loses the history that explains past results.
- `d`: Without links, readers find the old entry and think it still applies.
- `c|r2`: Right answer, wrong reason. The point is preserving reasoning, not storage.

B18-L3 exercise 2

Hints. (1) Which entries are accepted? (2) Does the superseded entry point to an existing one? (3) Yes: D-007.

Answer.

```
['D-007'] True
```

Current decisions are the accepted ones. Every superseded entry should point to an existing replacement.

Common wrong answers.

- `['D-003', 'D-007'] True`: D-003 is superseded, so it is not a current decision.

B18-L3 exercise 3

Hints. (1) Problem first. (2) Then the choices. (3) Then the pick and its cost.

Answer.

1. ID and title
2. Date and status
3. Context: the problem and the evidence
4. Options considered
5. Decision
6. Consequences and review point

Context, options, decision, consequences: the order of the reasoning itself.

Common wrong answers.

- State the problem and options before the choice, so the reader can judge it.
- Options come before the decision that chooses among them.

B18-L3 exercise 4

Hints. (1) Use entry.get(...) so missing keys do not raise. (2) Check the rules in the order given and append a message for each failure. (3) For superseded entries, test membership in log_ids.

Answer.

```
VALID_STATUS = {"proposed", "accepted", "superseded"}


def decision_problems(entry: dict[str, object], log_ids: set[str]) ->
    list[str]:
    """Problems with one decision log entry, in rule order.
```

```
    Args:
        entry: Parsed decision entry.
        log_ids: IDs of all entries in the log.

    Returns:
        Problem messages. Empty when the entry is complete.
    """
    problems = []
    status = entry.get("status")
    if status not in VALID_STATUS:
        problems.append("invalid status")
    options = entry.get("options")
    if not isinstance(options, list) or len(options) < 2:
        problems.append("fewer than two options")
    consequences = entry.get("consequences")
    if not isinstance(consequences, str) or not consequences.strip():
        problems.append("no consequences")
    if status == "superseded" and entry.get("superseded_by") not in
log_ids:
        problems.append("superseded without a valid replacement")
    return problems
```

A validator in CI keeps the log useful: every entry has real options and consequences, and every superseded entry leads somewhere.

B18-L3 exercise 5

Hints. (1) Where does the next analyst look? (2) When are the reasons clearest? (3) How does a reader meet the decision?

Answer.

- In the repository, versioned with the code it explains.
- Written with the change, while the reasons are fresh.
- Linked from where people meet the decision.

Records are useful when they are versioned, timely and linked.

Common wrong answers.

- where = `slides`: Slides are hard to find, hard to search and not versioned with the code.
- when = `later`: By the end, the options and reasons are half forgotten.
- link = `search`: People do not search for a record they do not know exists. Link it from the value.